ASIAN SCHOOL OF BUSINESS

Programme	**:**	**BCA (Honours) AI and Data Science**
Course Name	**:**	**Database Management Systems**
Course Code	**:**	25-9222-304
Given Date	**:**	1-10-26
Date of Submission	**:**	12-10-26

Assignment - 3

Topic: Seller And Inventory Tables

Submitted to,

Athira A G

Assistant Professor

Submitted by,

Name: Shivganga R T

Class: BCA

SKILL DEVELOPMENT WEEK 3

SELLER AND INVENTORY MANAGEMENT SYSTEM

INTRODUCTION

The **Seller** and **Inventory Management System** is a database project designed to manage product distribution and stock visibility using **MySQL**. Its objectives are to create **Seller** and **Inventory** tables, establish clean structural relationships between sellers, products, and physical stock, manage active seller product mappings, track available versus unavailable products, and generate inventory status reports for business analytics.

2. DATABASE DESIGN

The system consists of two tables **Seller** and **Inventory**.

2.1 Seller Table

Stores information about sellers.

Attribute	Data Type	Description
seller_id	INT	Primary key
seller_name	VARCHAR(100)	Seller name
email	VARCHAR(100)	Email address
phone	VARCHAR(20)	Contact number
address	VARCHAR(255)	Seller address

2.2 Inventory Table

Stores product details and stock information associated with each seller.

Attribute	Data Type	Description
inventory_id	INT	Primary key
seller_id	INT	Foreign key referencing Seller
product_name	VARCHAR(150)	Product name
category	VARCHAR(100)	Product category
price	DECIMAL(10,2)	Product price
quantity	INT	Available stock
reorder_level	INT	Minimum stock level

3. SQL IMPLEMENTATION

3.1 Create the Database

```
CREATE DATABASE SellerInventoryDB;
USE SellerInventoryDB;
```

3.2 Create the Seller Table

```
CREATE TABLE Seller (seller_id INT PRIMARY KEY,
seller_name VARCHAR(100) NOT NULL, email VARCHAR(100) UNIQUE,
phone VARCHAR(20), address VARCHAR(255));
```

3.3 Create the Inventory Table

```
CREATE TABLE Inventory ( inventory_id INT PRIMARY KEY, seller_id INT,
product_name VARCHAR(150), category VARCHAR(100), price DECIMAL(10,2),
quantity INT, reorder_level INT, last_updated TIMESTAMP, FOREIGN KEY
(seller_id) REFERENCES Seller(seller_id) );
```

The seller_id foreign key establishes a one-to-many relationship: one seller can have multiple inventory records.

4. INSERT SAMPLE DATA

4.1 Insert Sellers

INSERT INTO Seller VALUES (1,'ABC Traders', 'abc@example.com', '9876543210', 'Kochi'), (2,'XYZ Supplies', 'xyz@example.com','9876501234', 'Thiruvananthapuram'), (3, 'Global Trading', 'global@email.com',’9497814257’, '789 Market Ave');

4.2 Insert Inventory Records

INSERT INTO Inventory VALUES (1,1, 'Laptop', 'Electronics', 55000.00, 15, 5), (2,1, 'Keyboard', 'Accessories', 1200.00, 0, 10), (3,2, 'Office Chair', 'Furniture', 4500.00, 8, 3);

5. MAINTAIN SELLER AND PRODUCT INFORMATION

5.1 Update Seller Details

UPDATE Seller SET phone = '9999988888' WHERE seller_id = 1;

5.2 Update Product Price

UPDATE Inventory SET price = 1250.00 WHERE inventory_id = 2;

5.3 Update Stock Quantity

UPDATE Inventory SET quantity = quantity + 10 WHERE inventory_id = 2;

These queries allow seller information, product prices, and inventory quantities to be updated when necessary.

6. TRACK PRODUCT AVAILABILITY

```
SELECT s.seller_name, i.product_name, i.quantity, i.reorder_level FROM Seller s
JOIN Inventory i ON s.seller_id = i.seller_id;
```

This query identifies whether each product is available, unavailable, or running low on stock.

7. INVENTORY STATUS REPORTS

7.1 Complete Inventory Report

```
SELECT s.seller_name, i.product_name, i.category, i.price, i.quantity, i.reorder_level
FROM Seller s JOIN Inventory i ON s.seller_id = i.seller_id ORDER BY
s.seller_name, i.product_name;
```

7.2 Unavailable Products Report

```
SELECT s.seller_name, i.product_name, i.quantity FROM Seller s JOIN Inventory i
ON s.seller_id = i.seller_id WHERE i.quantity = 0;
```

7.3 Low-Stock Report

```
SELECT s.seller_name, i.product_name,i.quantity,i.reorder_level FROM Seller s JOIN
Inventory i ON s.seller_id = i.seller_id WHERE i.quantity > 0
AND i.quantity <= i.reorder_level;
```

7.4 Inventory Summary Report

```
SELECT COUNT(*) AS total_products, SUM(quantity > 0) AS available_products,
SUM(quantity = 0) AS unavailable_products, SUM(quantity) AS total_units
FROM Inventory;
```

For the sample data, the summary reports three inventory records, two available products, one unavailable product, and 23 units in stock.

CONCLUSION

The **Seller** and **Inventory Management System** provides a simple and structured method for maintaining seller details and inventory records using **MySQL**. The two-table design establishes relationships through primary and foreign keys, supports product and stock updates, tracks product availability, and generates inventory status reports. This project demonstrates the practical use of relational database concepts in inventory management.